Single-Cell RNA-seq Analysis Pipeline

From Raw Reads to Differential Expression and GO Enrichment

2026-03-31

Contents

1 Overview **3**

2 Requirements **4**
- 2.1 Hardware . . . 4
- 2.2 Docker installation . . . 4
  - 2.2.1 Linux (Ubuntu/Debian) . . . 4
  - 2.2.2 macOS / Windows . . . 4
- 2.3 Pulling the container . . . 4
- 2.4 Starting an interactive session . . . 4

3 Step 1: Reference Genome Preparation **5**
- 3.1 Downloading the genome and annotation . . . 5
- 3.2 Filtering to protein-coding genes . . . 5
- 3.3 Building the CellRanger reference index . . . 5

4 Step 2: Read Quantification (CellRanger) **6**
- 4.1 FASTQ file naming convention . . . 6
- 4.2 Running cellranger count . . . 6

5 Step 3: Ambient RNA Removal (CellBender) **7**

6 Step 4: Quality Control **8**
- 6.1 Pre-filter assessment . . . 8
- 6.2 Filtering parameters . . . 8
- 6.3 Post-filter confirmation . . . 8

7 Step 5: Data Merging, Normalisation, and Preprocessing **11**

8 Step 6: Batch Correction (Harmony) **12**
- 8.1 Choosing the number of dimensions . . . 12

9 Step 7: Clustering Resolution Selection (Clustree) **13**

10 Step 8: Final Clustering **15**

11 Step 9: Cell Count per Cluster **17**

12 Step 10: Global Pseudobulk Replicate Correlation **19**

13 Step 11: Marker Gene Identification and Cell-Type Annotation **20**
13.1 Marker genes 20
13.2 Reference-based annotation 20
13.3 Annotated clustree 20

14 Step 12: Gene Expression Visualisation **22**

15 Step 13: Cell-Type Grouping and Manual Curation **24**
15.1 Grouping fine annotations 24
15.2 Manual subclustering and curation 24

16 Step 14: Marker Gene Dot Plot **26**

17 Step 15: Export to Python (h5ad) **27**

18 Step 16: Pseudobulk per Cell Type and DESeq2 Differential Expression **28**
18.1 Pseudobulk aggregation 28
18.2 DESeq2 analysis 28
18.3 Volcano plots 28
18.4 Differential expression heatmap 28

19 Step 17: GO Enrichment Analysis **29**
19.1 Organism configuration 29

20 Adapting the Pipeline to Other Organisms **30**

21 Output File Summary **31**

1 Overview

This document describes a complete, reproducible computational pipeline for analysing single-cell RNA sequencing (scRNA-seq) data generated on the 10x Genomics Chromium platform. The workflow starts from raw FASTQ files, aligns reads to a reference genome, removes ambient RNA contamination, performs quality control, integrates multiple samples by correcting batch effects, clusters cells, assigns cell-type identities, and finally performs pseudobulk differential expression and gene ontology (GO) enrichment analysis.

All software runs inside a Docker container (`mvergara19/scrnaseq_docker:latest`), which packages R 4.5, Python 3, Seurat 5, harmony, DESeq2, CellRanger 9.0.1, CellBender, and all dependencies into a single reproducible environment. No local R or Python installation is needed beyond Docker itself.

The pipeline is **organism-agnostic**. The examples throughout this document use *Arabidopsis thaliana* under three nitrogen conditions (0N, 0.5N, and 5N), but adapting it to human, mouse, or any other species requires changing only three parameters: the mitochondrial gene prefix, the GO annotation database, and the key type.

---

2 Requirements

2.1 Hardware

Resource	Minimum	Recommended
RAM	16 GB	64 GB (datasets $>$ 30,000 cells)
Storage	50 GB free	200 GB
CPU	4 cores	24 cores (for CellRanger)

2.2 Docker installation

2.2.1 Linux (Ubuntu/Debian)

```
sudo apt-get update && sudo apt-get install -y docker.io
sudo systemctl enable --now docker
sudo usermod -aG docker $USER   # log out and back in after this
```

2.2.2 macOS / Windows

Download Docker Desktop from `https://www.docker.com/products/docker-desktop/` and follow the installer. On Windows, WSL 2 is required and the installer guides the setup.

2.3 Pulling the container

```
docker pull mvergara19/scrnaseq_docker:latest
```

2.4 Starting an interactive session

```
docker run --rm -it -v $(pwd):/workspace mvergara19/scrnaseq_docker:latest bash
```

The `-v` flag mounts the current directory into `/workspace` inside the container. All input data and output files should live under this directory.

---

3 Step 1: Reference Genome Preparation

Before quantifying gene expression, a species-specific reference must be built from two files: a genome sequence (`.fa`) and a gene annotation (`.gtf`). This step is performed once per organism.

3.1 Downloading the genome and annotation

For *Arabidopsis thaliana* (Ensembl Plants release 53):

```
wget http://ftp.ensemblgenomes.org/pub/plants/release-53/fasta/arabidopsis_thaliana/dna/Arabido

wget http://ftp.ebi.ac.uk/ensemblgenomes/pub/release-53/plants/gtf/arabidopsis_thaliana/Arabido

gzip -d Arabidopsis_thaliana.*
```

For **human** or **mouse**, pre-built CellRanger references are available at the 10x Genomics support website.

3.2 Filtering to protein-coding genes

CellRanger recommends restricting the annotation to protein-coding genes to reduce multi-mapping and improve accuracy:

```
cellranger mkgtf \
    Arabidopsis_thaliana.TAIR10.53.gtf \
    Arabidopsis_thaliana.TAIR10.53.filtrado.gtf \
    --attribute=gene_biotype:protein_coding
```

3.3 Building the CellRanger reference index

```
cellranger mkref \
    --genome=Arabidopsis_thaliana.TAIR10.53.transcriptome \
    --fasta=Arabidopsis_thaliana.TAIR10.dna.toplevel.fa \
    --genes=Arabidopsis_thaliana.TAIR10.53.filtrado.gtf
```

This generates an indexed reference directory used by all subsequent `cellranger count` calls. Building the reference typically takes 20–40 minutes.

---

4 Step 2: Read Quantification (CellRanger)

CellRanger aligns cDNA reads to the reference genome using STAR, corrects cell barcodes, collapses UMIs, and produces a filtered count matrix in which each column is a cell and each row is a gene.

4.1 FASTQ file naming convention

Each 10x Genomics library produces three FASTQ files per sequencing lane:

Suffix	Content
`*_I1_001.fastq`	Sample index (8 bp)
`*_R1_001.fastq`	Cell barcode (16 bp) + UMI (12 bp)
`*_R2_001.fastq`	cDNA read (the actual transcript sequence)

For example, a library with accession `SRR8485805` would have files named:

```
SRR8485805_S1_L001_I1_001.fastq
SRR8485805_S1_L001_R1_001.fastq
SRR8485805_S1_L001_R2_001.fastq
```

4.2 Running cellranger count

```
cellranger count \
    --localcores=24 \
    --id=run_count \
    --fastqs=/workspace/fastqs \
    --sample=SRR8485805 \
    --transcriptome=/workspace/Arabidopsis_thaliana.TAIR10.53.transcriptome
```

Replace `--sample` with the filename prefix of your FASTQ files, and `--transcriptome` with the path to the reference built in Step 1. This step takes several hours per sample. The key output is `run_count/outs/raw_feature_bc_matrix.h5` (required by CellBender) and `run_count/outs/filtered_feature_bc_matrix/` (MEX format, for direct use without CellBender).

---

5 Step 3: Ambient RNA Removal (CellBender)

During library preparation, lysed cells release mRNA into the cell suspension. This ambient RNA is captured by empty droplets and inflates the counts of certain genes across all cells. CellBender uses a deep generative model to estimate the ambient RNA profile and subtract it from each cell's count vector.

```
cellbender remove-background \
    --input  /workspace/raw_feature_bc_matrix.h5 \
    --output /workspace/Sample_cellbender_output.h5
```

The corrected output (`Sample_cellbender_output_filtered.h5`) is the recommended input for all downstream R analyses. Running CellBender on a GPU significantly reduces computation time; to enable GPU support, launch the container with `--gpus all` and ensure the NVIDIA Container Toolkit is installed on the host.

---

6 Step 4: Quality Control

Quality control (QC) identifies and removes low-quality cells before downstream analysis. Three metrics are computed per cell:

- **nFeature_RNA** — number of distinct genes detected. Very low values indicate empty droplets; unusually high values may indicate doublets.
- **nCount_RNA** — total UMI count; correlated with sequencing depth.
- **percent.mt / percent.cp** — percentage of reads mapping to mitochondrial (or chloroplast, for plants) genes. Elevated values indicate damaged cells that have lost cytoplasmic mRNA.

6.1 Pre-filter assessment

Violin plots of these metrics across all samples are generated before any filtering. These plots guide the choice of appropriate thresholds.

6.2 Filtering parameters

Based on the pre-filter distributions, thresholds are applied:

- **Minimum genes per cell**: 200 (removes empty droplets)
- **Maximum mitochondrial %**: 5% (removes damaged cells; adjust per organism and tissue)

Additionally, DoubletFinder is applied to each sample individually to detect co-encapsulated cell pairs (doublets), which are removed before merging.

6.3 Post-filter confirmation

After filtering, the same violin plots are regenerated to confirm that low-quality cells have been removed and the distributions are clean.

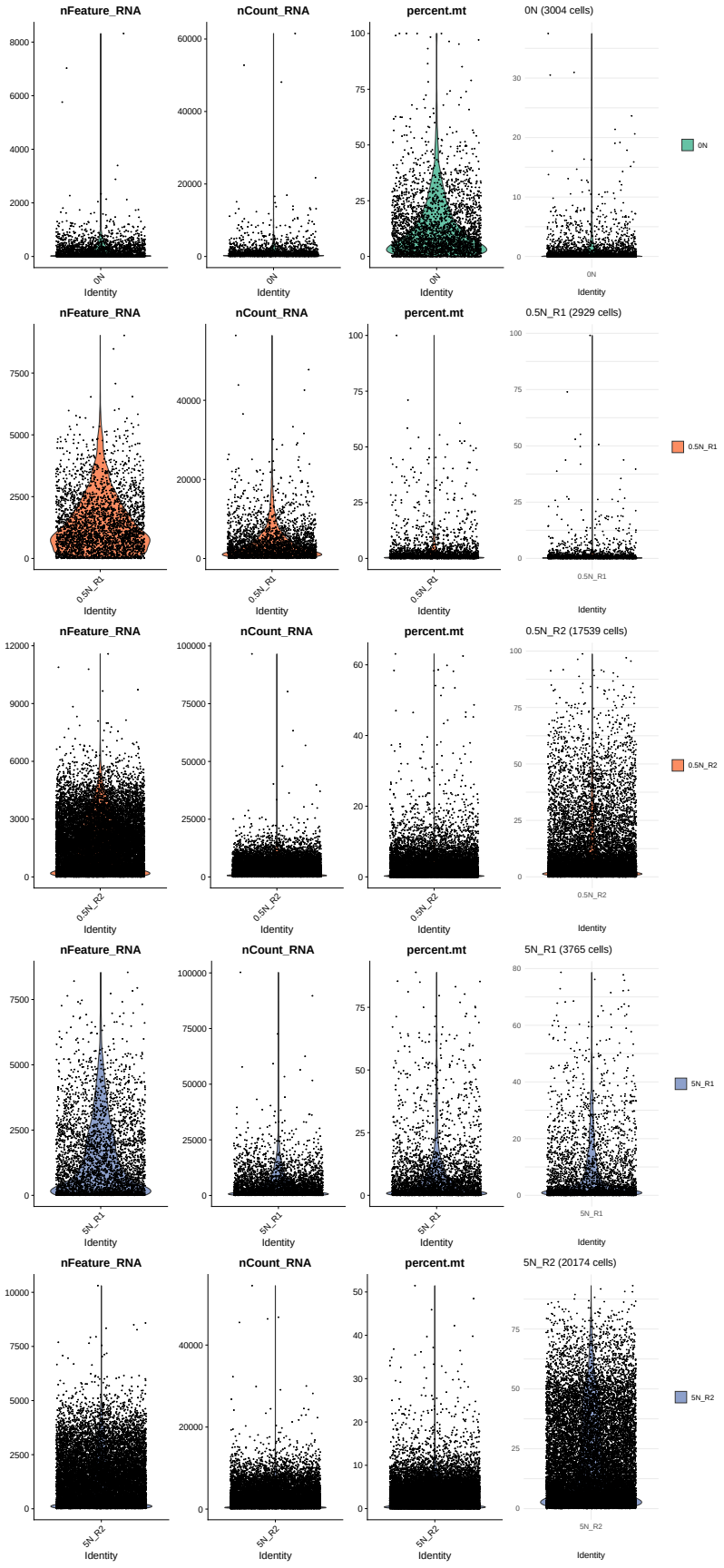

Figure 1: Pre-filter QC metrics. Each panel shows one sample. Violin plots display the distribution of detected genes (nFeature_RNA), total UMI count (nCount_RNA), and mitochondrial/chloroplast gene fractions before any filtering.

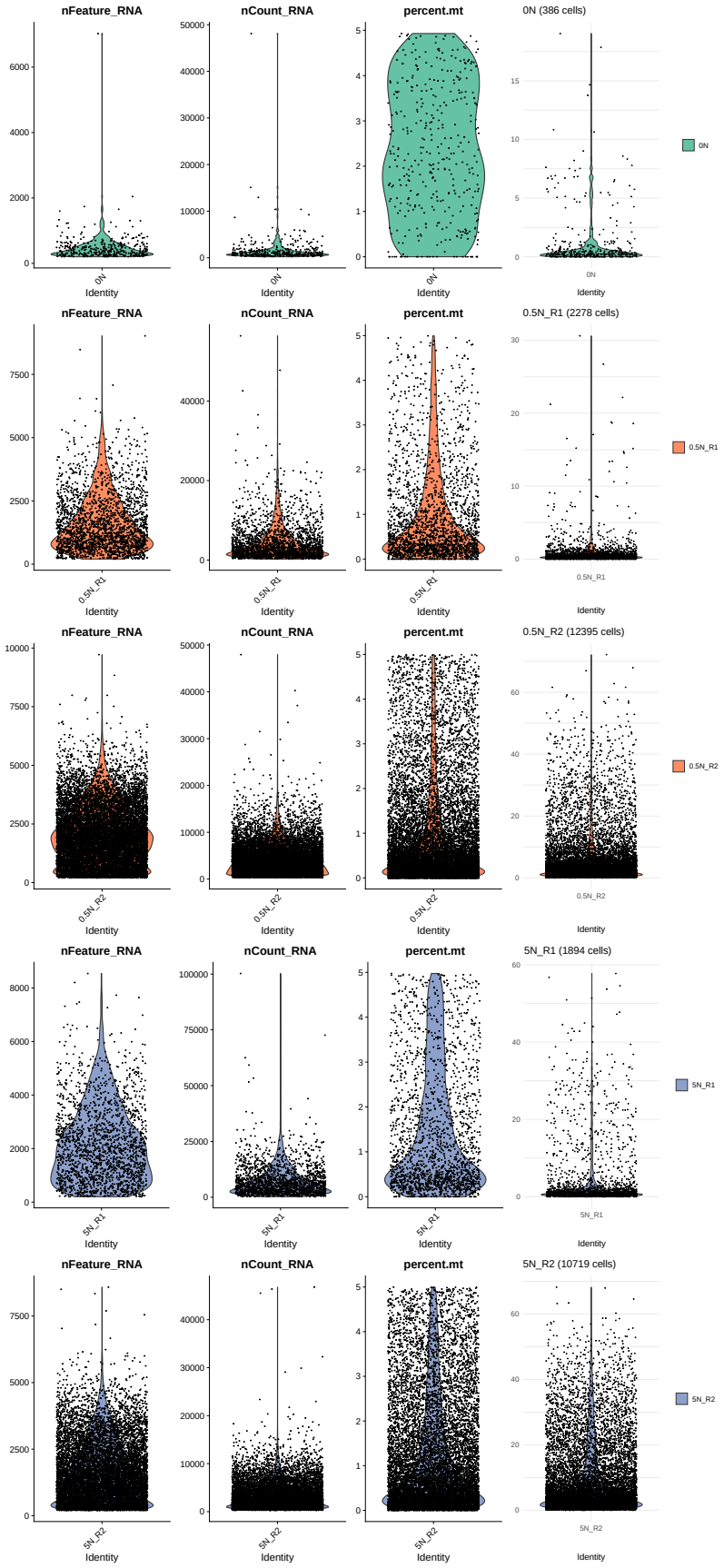

Figure 2: Post-filter QC metrics. The same violin layout as above, after applying the minimum-feature and maximum-mitochondrial-percent thresholds and removing DoubletFinder-predicted doublets.

7 Step 5: Data Merging, Normalisation, and Preprocessing

After quality control, all samples are merged into a single Seurat object. The following preprocessing steps are applied in sequence:

1. **Log-normalisation** — counts are normalised to 10,000 UMIs per cell and log-transformed to stabilise variance.
2. **Highly variable features** — the 2,000 most variable genes are selected for dimensionality reduction.
3. **Scaling** — expression values are centred and scaled so that highly expressed genes do not dominate PCA.
4. **PCA** — the top 30 principal components are computed from the variable features.
5. **UMAP** — a two-dimensional embedding is computed from PCA space for visualisation.

The UMAP below shows all cells before batch correction, coloured by sample. Separation of cells by sample (rather than by biology) confirms the presence of batch effects that must be corrected.

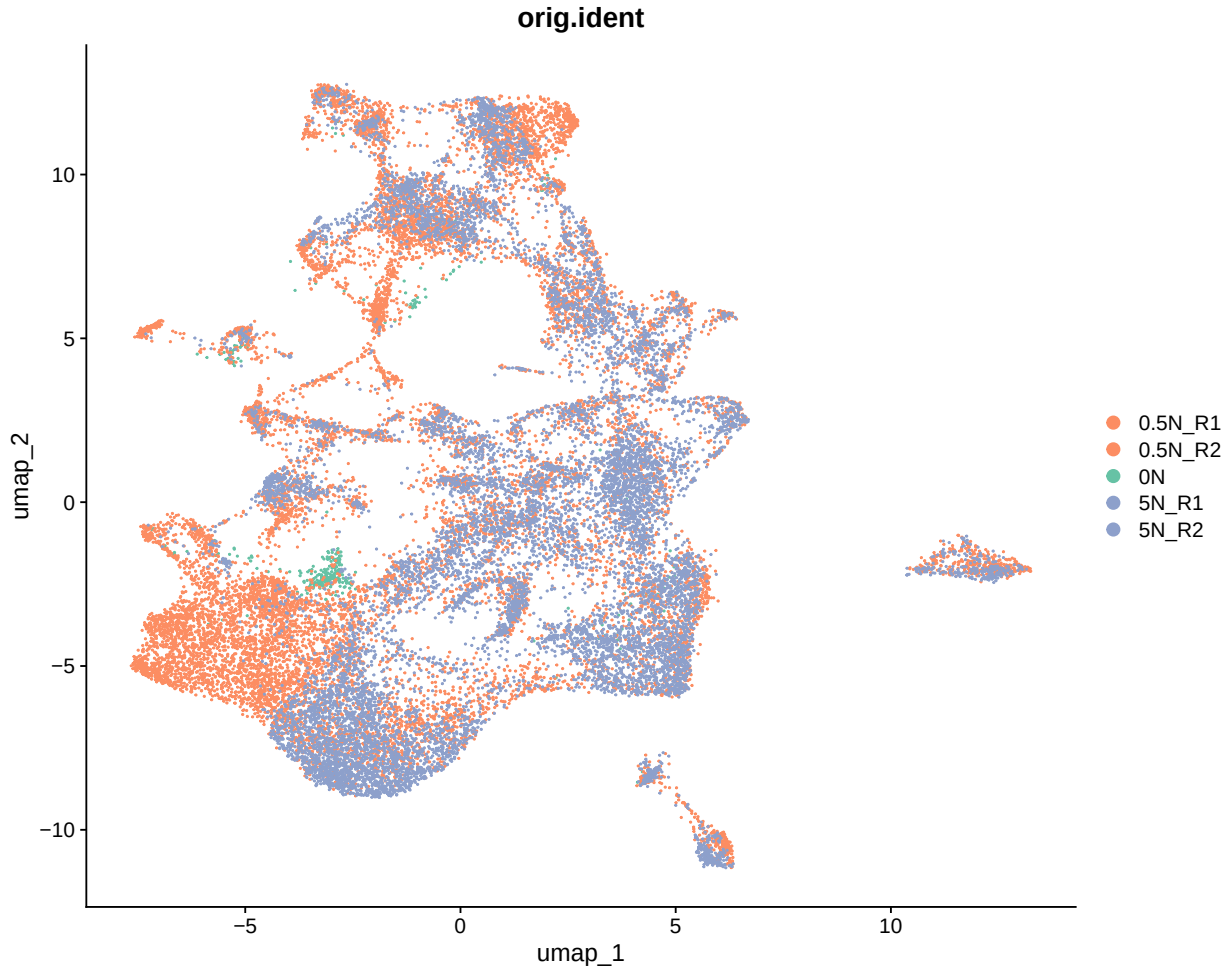

Figure 3: UMAP before Harmony batch correction, coloured by sample identity. Samples separate into distinct groups, reflecting technical batch effects.

8 Step 6: Batch Correction (Harmony)

Harmony iteratively adjusts the PCA embedding to remove sample-specific shifts while preserving biological signal. It integrates all samples by their `orig.ident` label and produces a corrected `harmony` reduction used for all downstream graph construction and UMAP embedding.

8.1 Choosing the number of dimensions

Before clustering, the number of significant principal components is assessed using a k-means within-cluster sum-of-squares (WSS) elbow plot. The optimal number of dimensions corresponds to the point where the curve flattens, indicating diminishing returns from additional components.

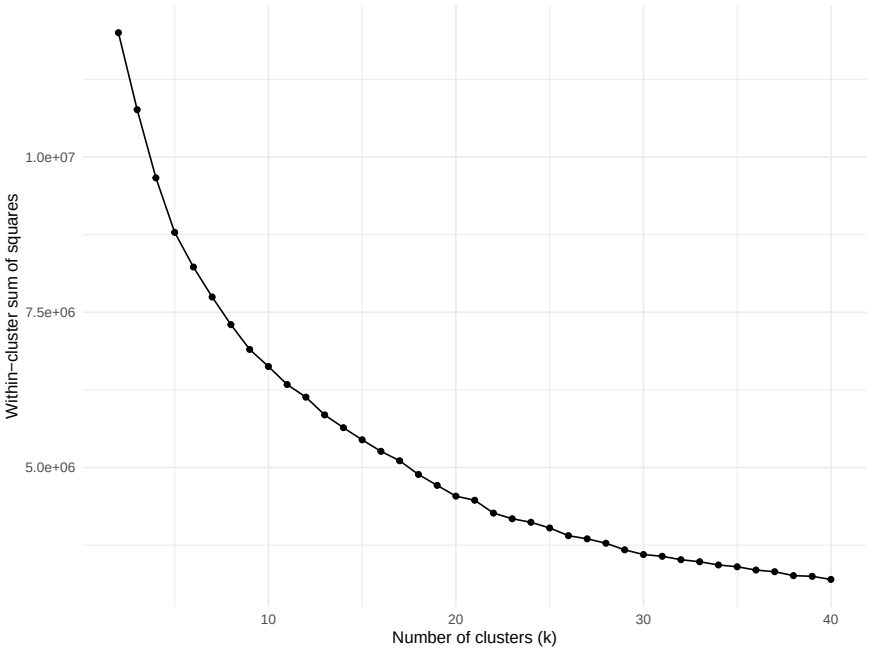

Figure 4: Elbow plot (k-means WSS) for dimension selection. The inflection point suggests that 30 dimensions capture the majority of meaningful biological variance.

9 Step 7: Clustering Resolution Selection (Clustree)

Seurat's graph-based clustering (Leiden algorithm) requires a resolution parameter that controls the granularity of the output. To select a stable resolution, clustering is run across a range of values (0.15, 0.30, 0.50, 0.80, 1.00) and visualised as a tree with the `clustree` package. Each node represents a cluster at a given resolution; edges indicate the fraction of cells flowing from one cluster to another as resolution increases. A stable resolution produces wide, consistent edges without excessive fragmentation.

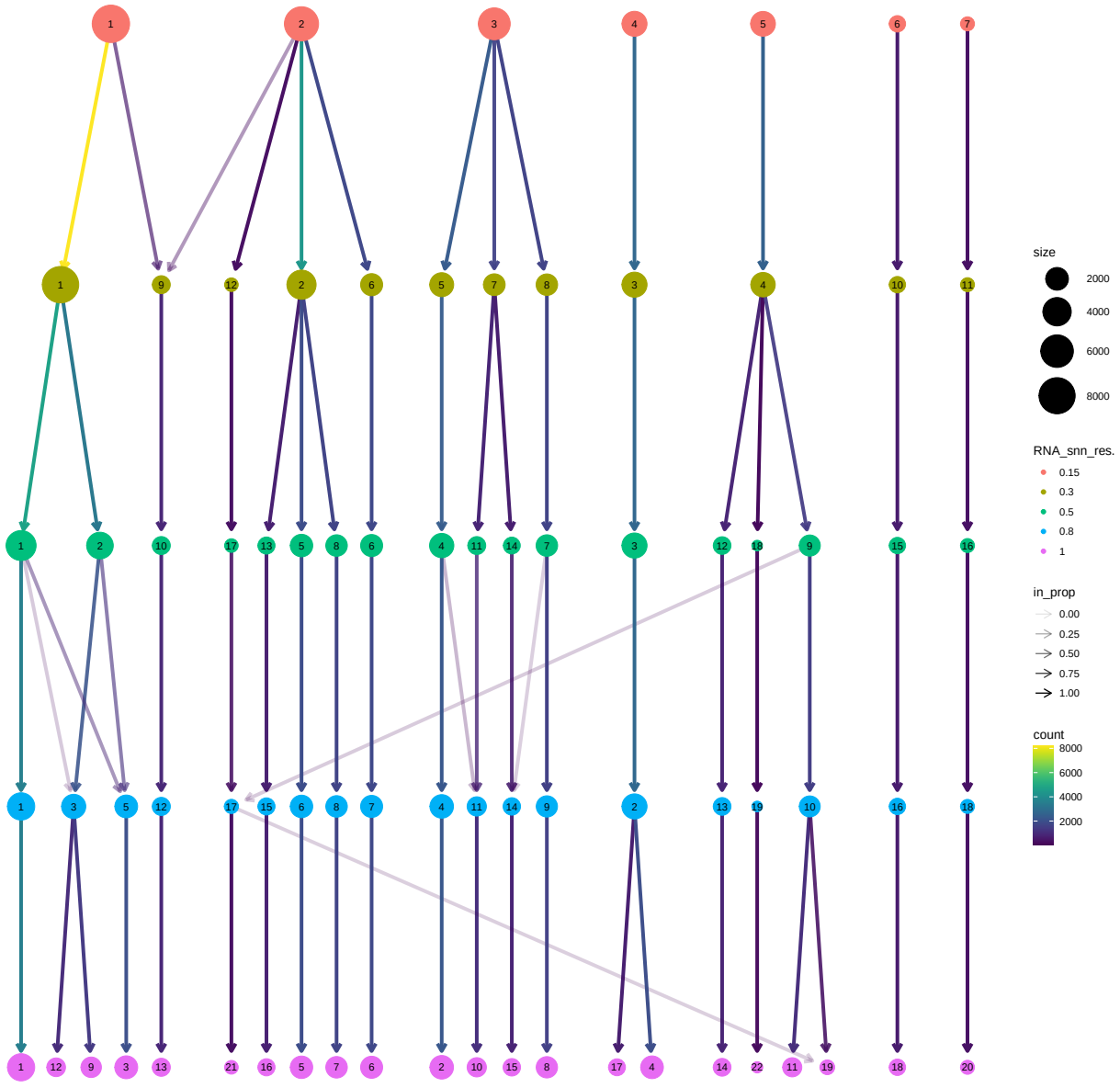

Figure 5: Clustree resolution sweep. Nodes represent clusters at each resolution; edge width is proportional to the number of cells carried over. A resolution of 0.3 was selected based on the stability of the partition.

10 Step 8: Final Clustering

Based on the clustree output, a resolution of 0.3 is applied for the final clustering. The resulting partition is visualised on the post-Harmony UMAP, coloured first by sample identity (to verify batch correction) and then by cluster identity.

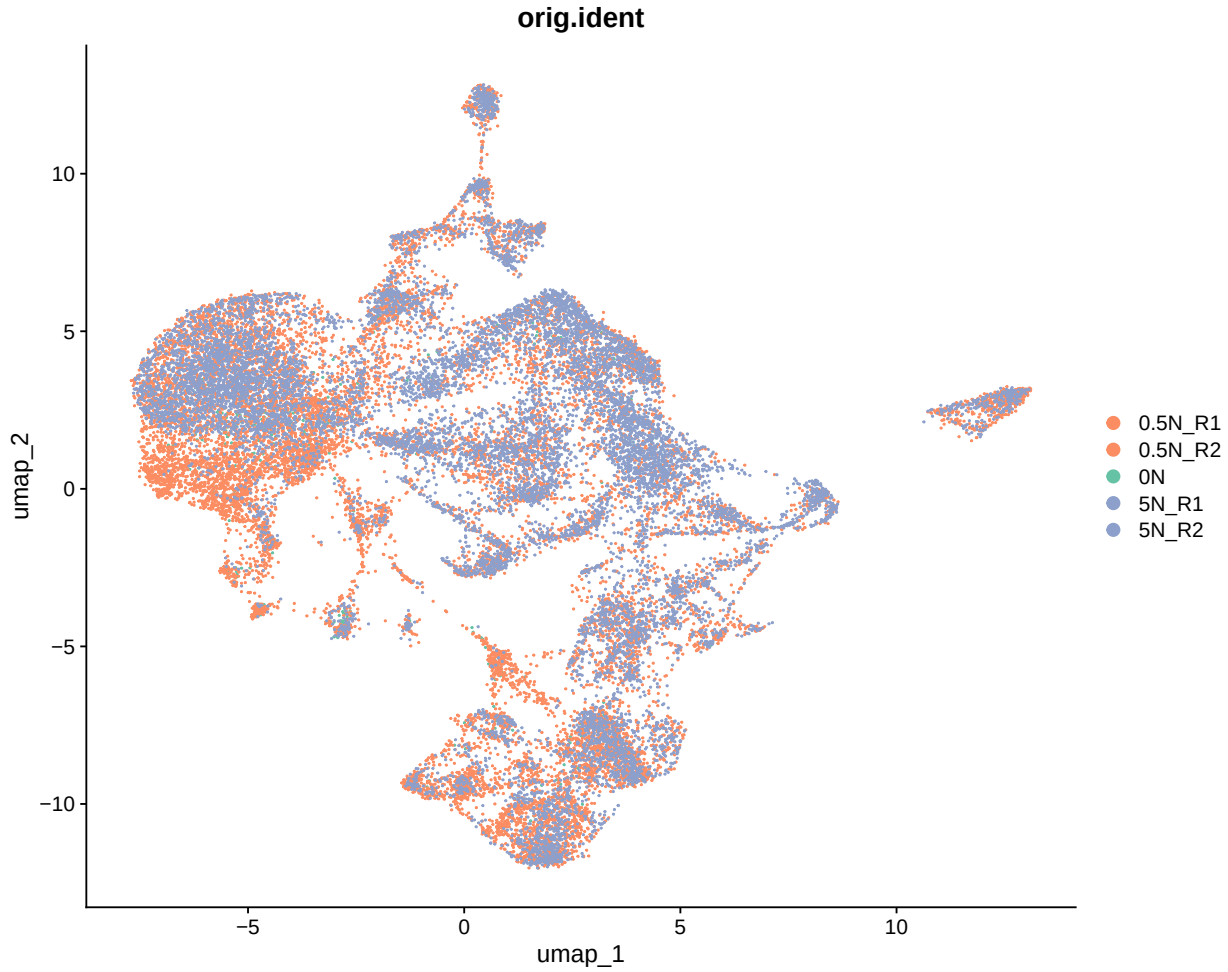

Figure 6: UMAP after Harmony batch correction, coloured by sample identity. Samples are now well mixed within each cluster, confirming successful batch correction.

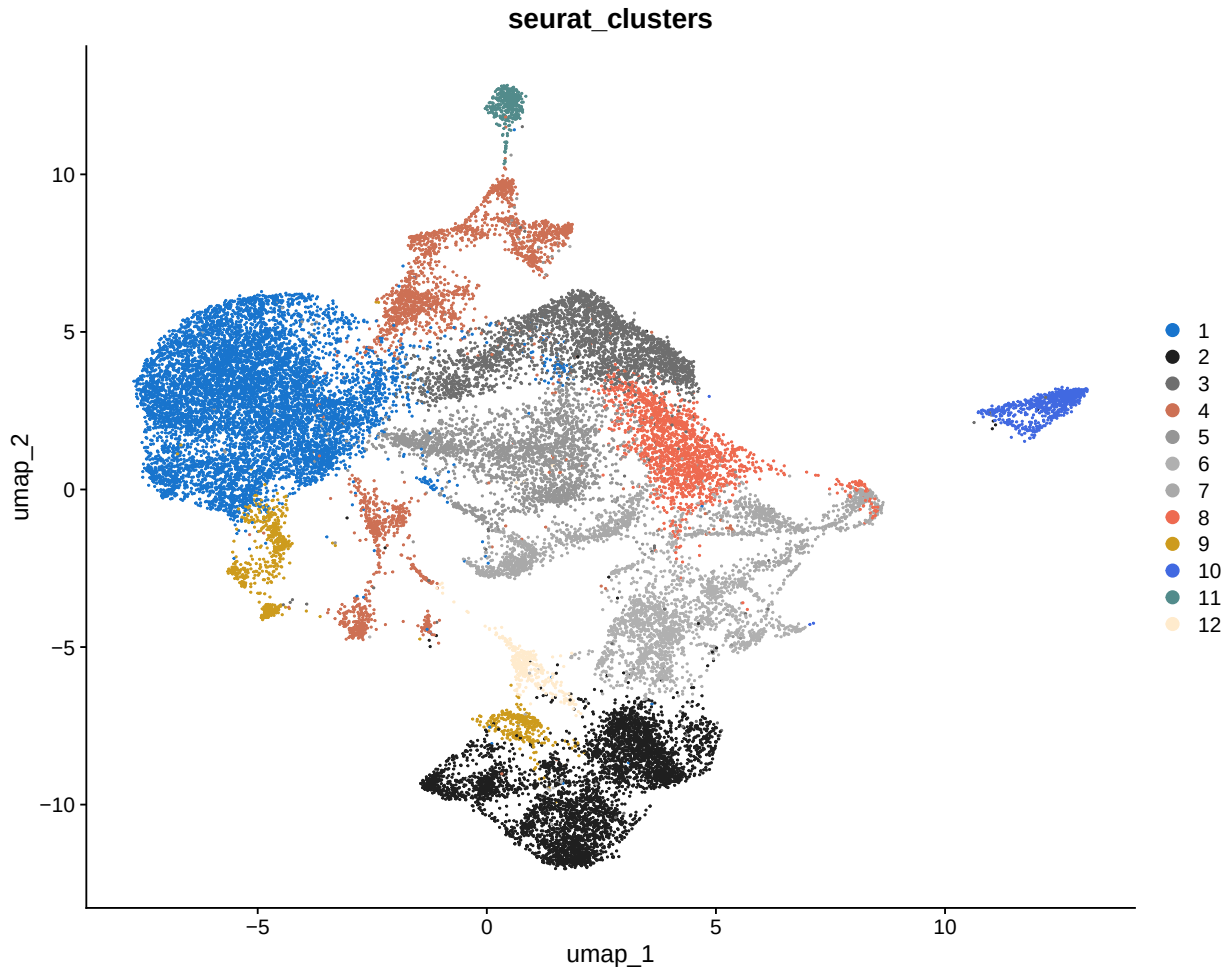

Figure 7: UMAP coloured by Seurat cluster identity at resolution 0.3.

11 Step 9: Cell Count per Cluster

Bar charts summarise the composition of each cluster. The left panel shows the fraction of cells contributed by each sample (useful for identifying sample-specific clusters), and the right panel shows the total cell count per cluster on a log10 scale.

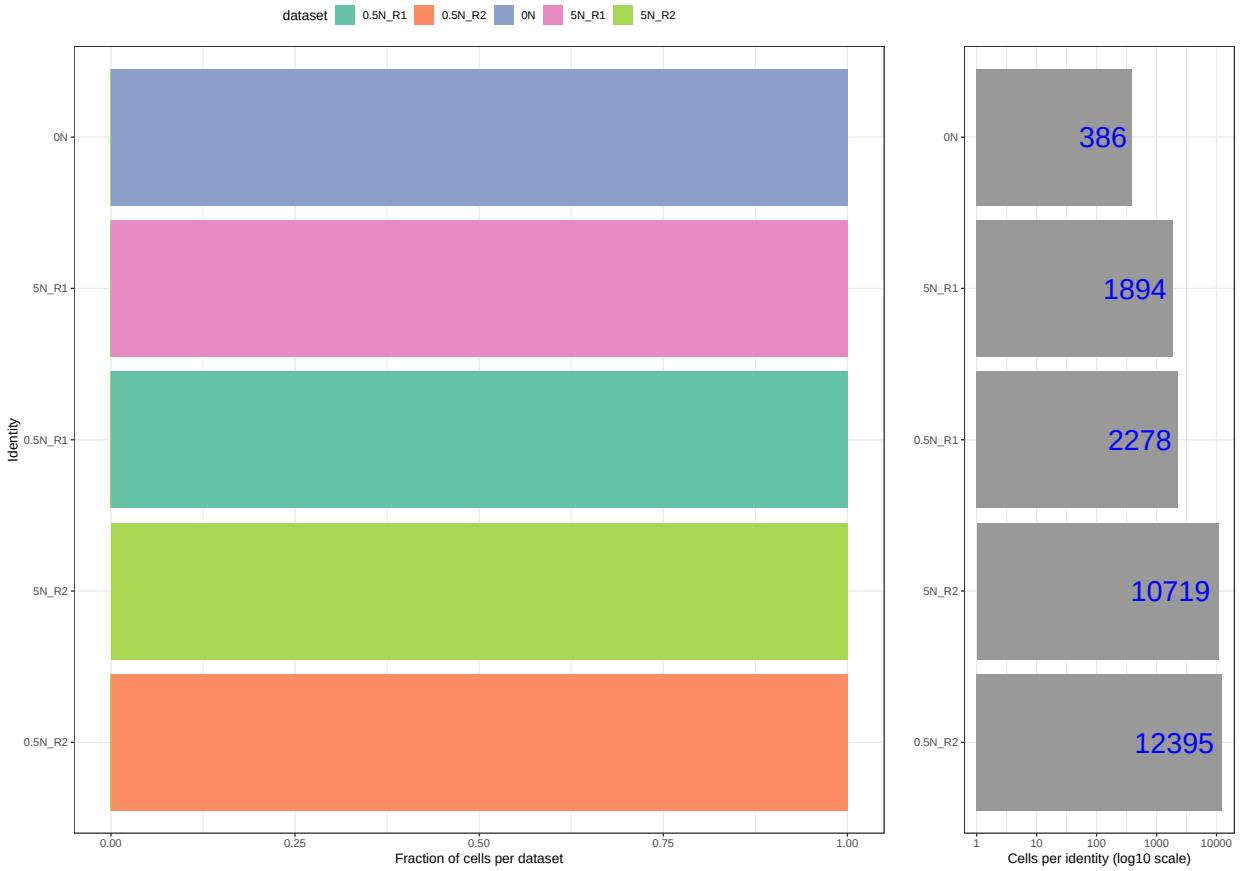

Figure 8: Cell count by sample identity. Left: fraction of cells from each sample per cluster. Right: total cells per cluster (log10 scale).

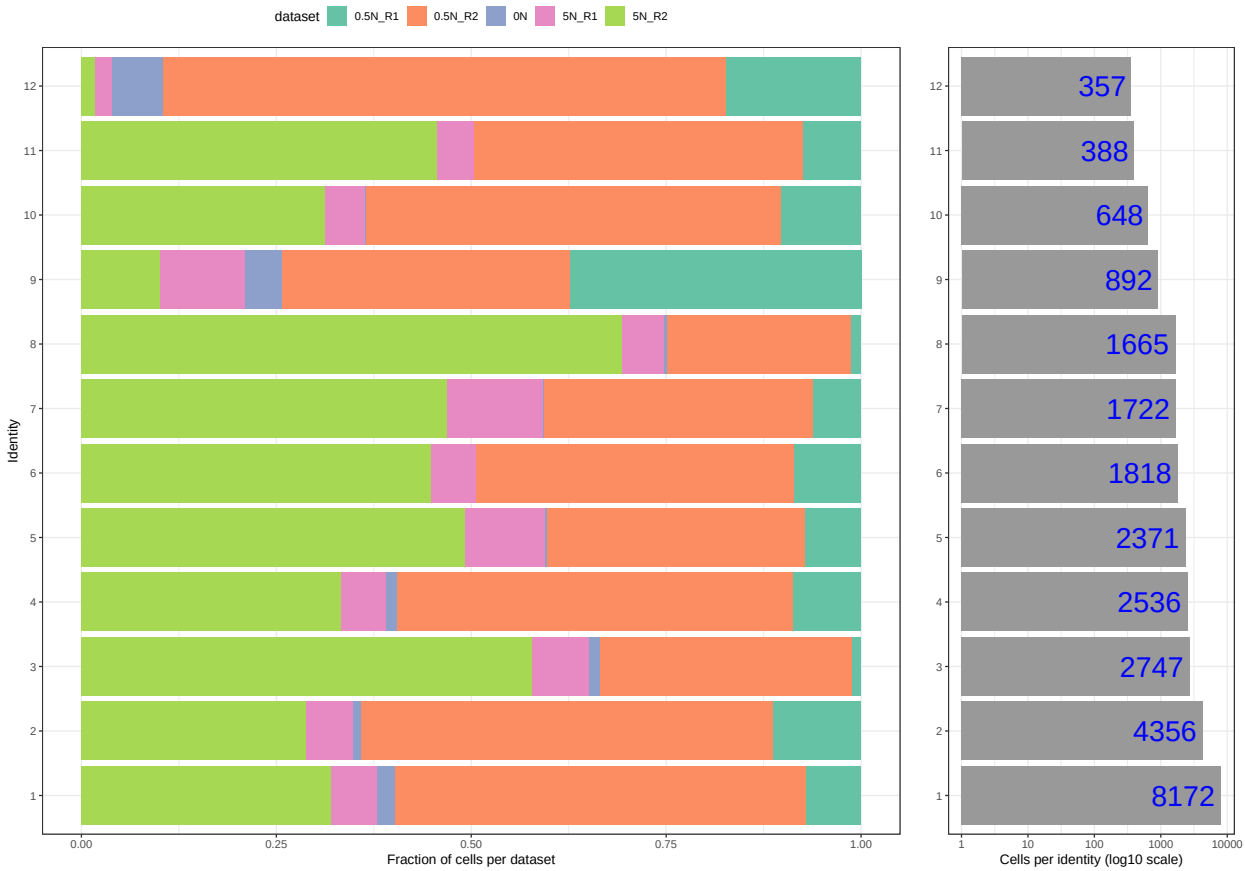

Figure 9: Cell count by Seurat cluster.

12 Step 10: Global Pseudobulk Replicate Correlation

A global quality check is performed by aggregating all cells per sample into a pseudobulk count vector and computing Pearson correlations between samples. Replicates from the same condition should cluster together with high correlation ($r > 0.95$). Outlier samples correlating poorly with their replicates may indicate technical problems.

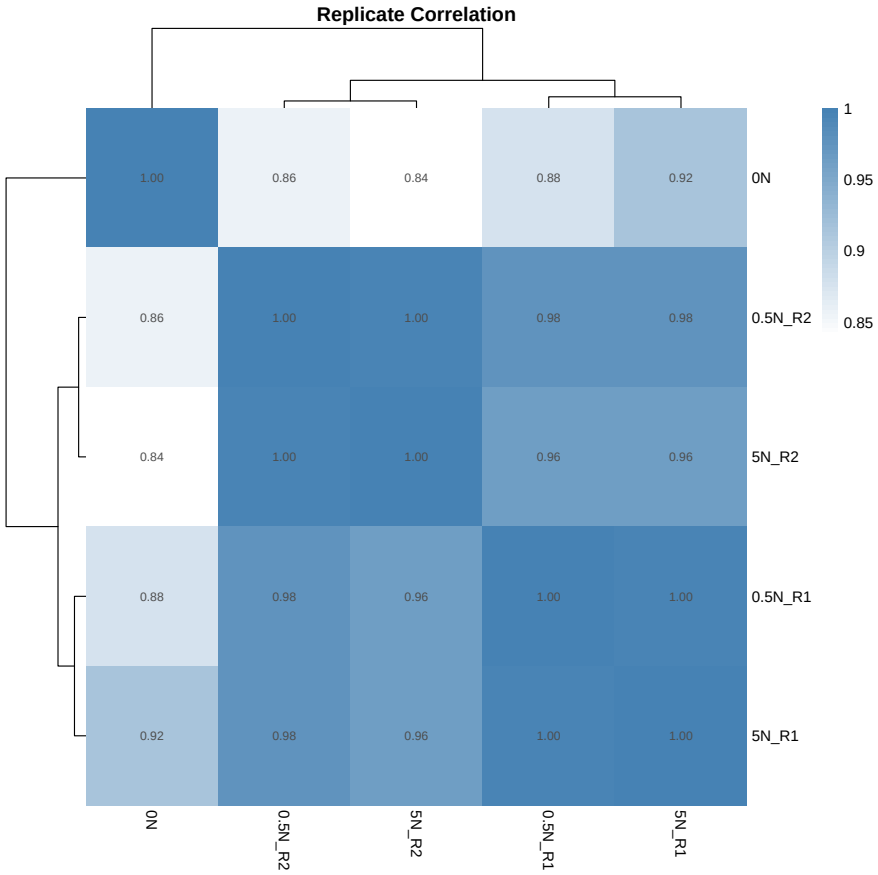

Figure 10: Pearson correlation heatmap of global pseudobulk profiles. Biological replicates within the same condition are expected to cluster together with high correlation.

13 Step 11: Marker Gene Identification and Cell-Type Annotation

13.1 Marker genes

For each cluster, genes significantly overexpressed relative to all other clusters are identified using a Wilcoxon rank-sum test (Seurat's `FindAllMarkers`), retaining genes expressed in at least 25% of cells with a log2 fold change above 0.25. The top markers per cluster are cross-referenced against curated cell-type gene signatures from the literature.

13.2 Reference-based annotation

Cell-type labels are also transferred from a published reference Seurat object using Seurat's `FindTransferAnchors` / `TransferData` workflow. This approach is robust when a high-quality reference atlas is available for the organism.

13.3 Annotated clustree

The resolution sweep from Step 7 is repeated on the annotated object, overlaying the most frequent cell-type label at each cluster node to confirm that the chosen resolution separates biologically distinct cell types.

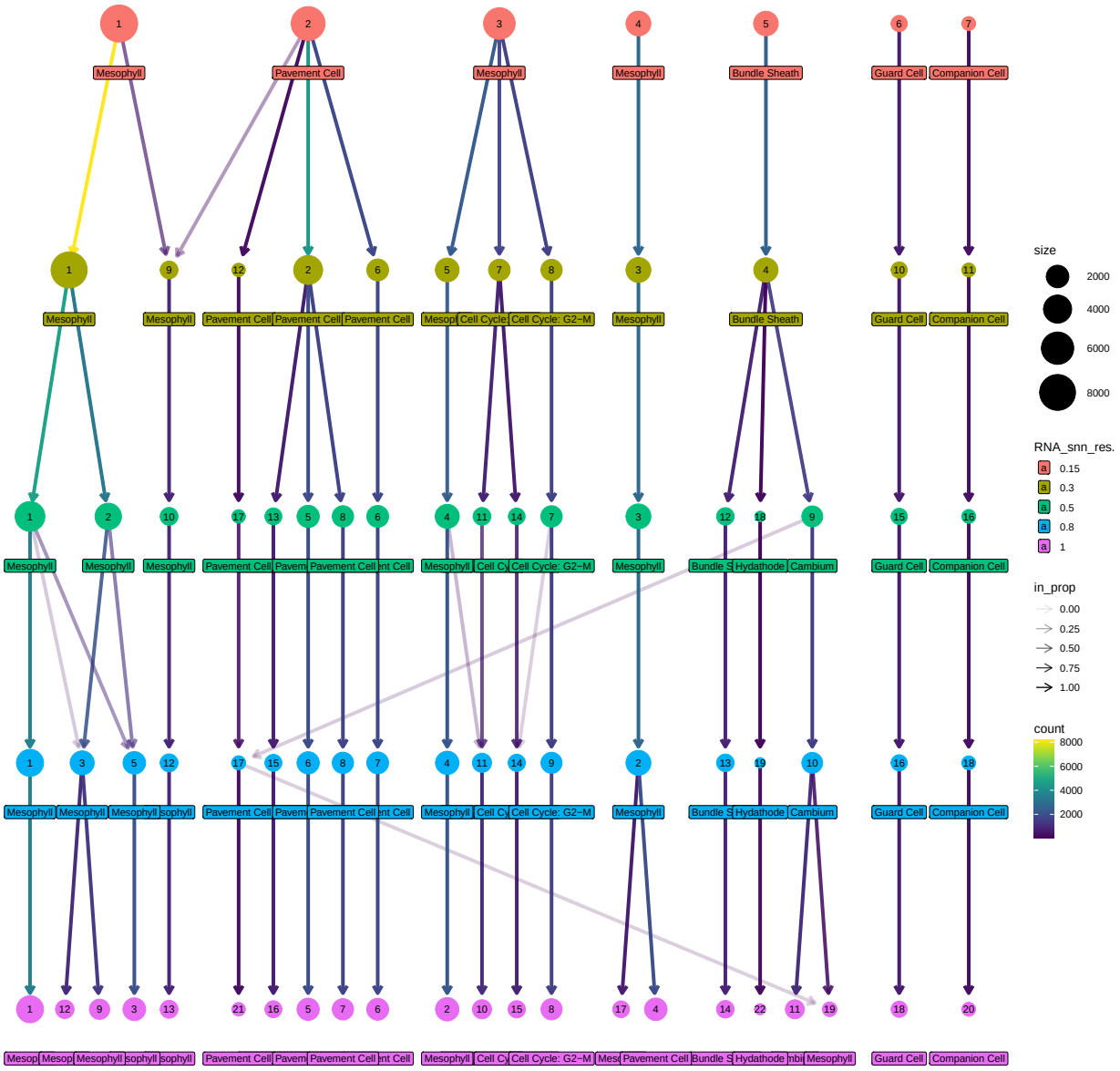

Figure 11: Annotated clustree. Node labels show the most frequent transferred cell-type label within each cluster at each resolution.

14 Step 12: Gene Expression Visualisation

Individual genes or gene sets can be inspected with violin plots (expression distribution per cell type) and feature plots (expression projected onto the UMAP). These can be generated globally across all cell types or restricted to a single cell type of interest.

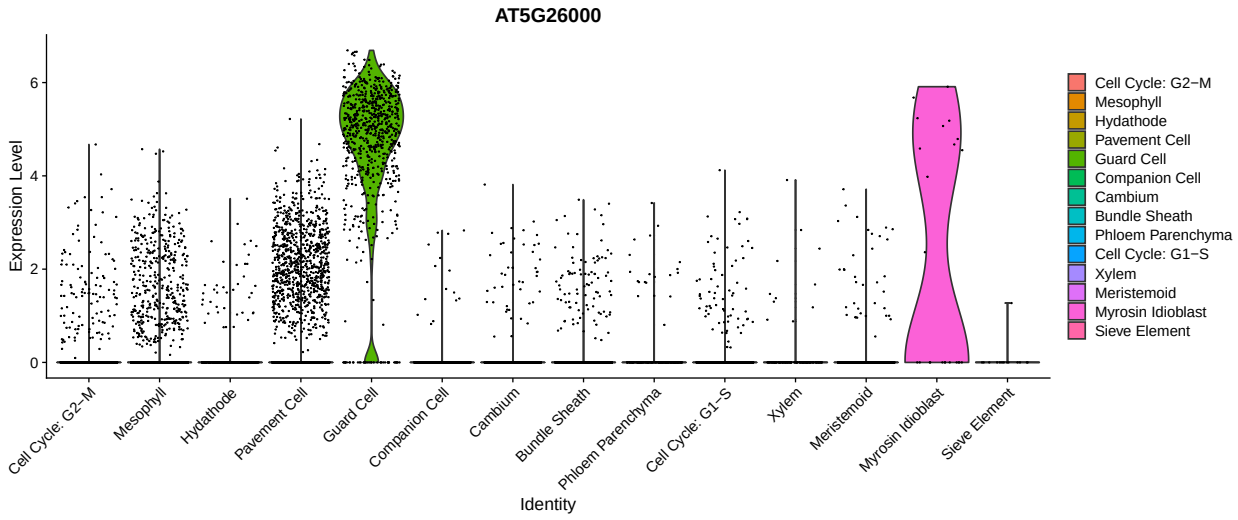

Figure 12: Violin plot of AT5G26000 expression across all annotated cell types.

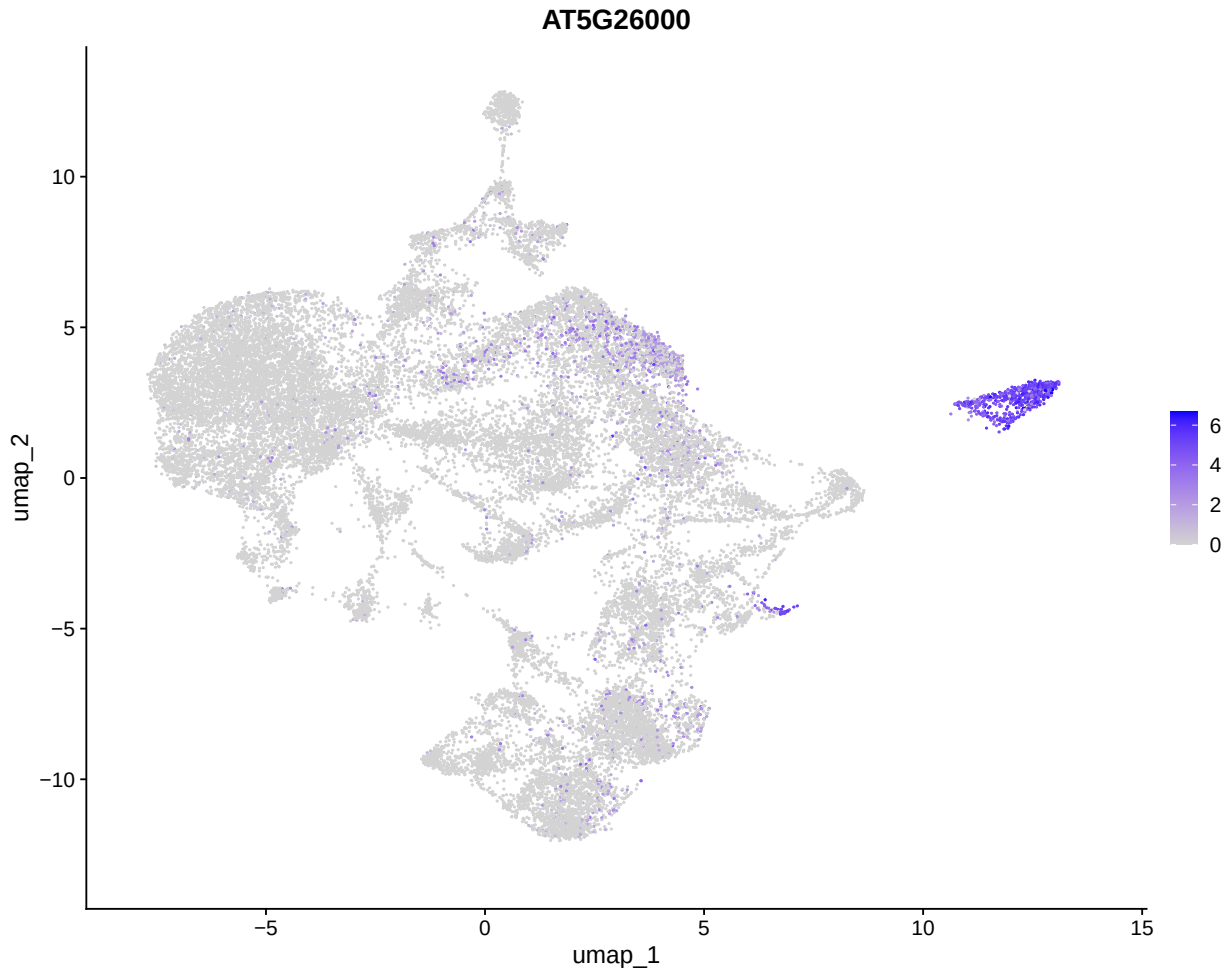

Figure 13: Feature plot of AT5G26000 projected onto the UMAP. Colour intensity indicates normalised expression level.

15 Step 13: Cell-Type Grouping and Manual Curation

15.1 Grouping fine annotations

Fine-grained cell-type labels are collapsed into broader biological categories via a configurable mapping table. For example, companion cells, cambium, phloem parenchyma, xylem, and sieve elements are grouped under "Vascular Cell" for comparisons that do not require fine-grained resolution.

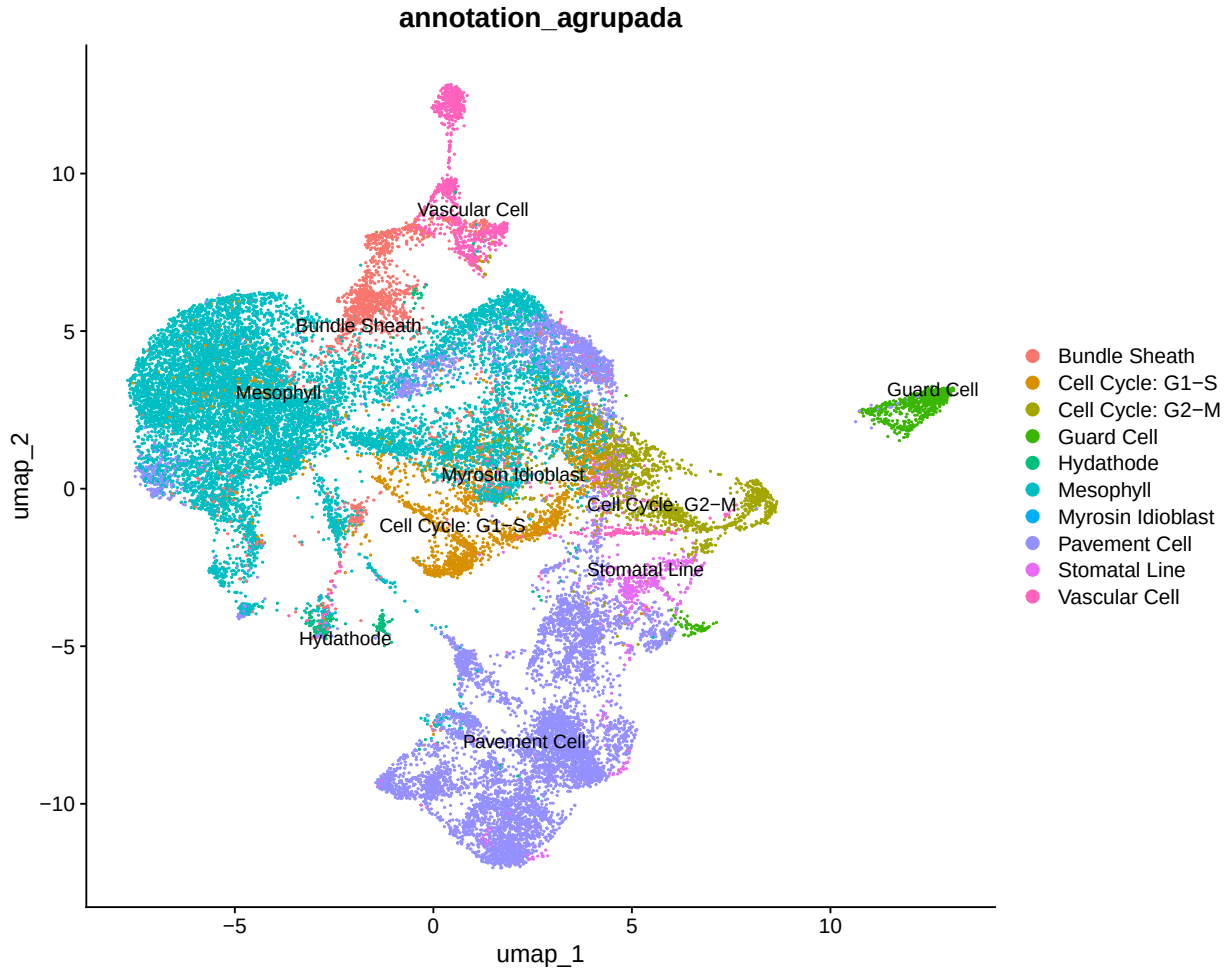

Figure 14: UMAP coloured by grouped cell-type annotations. Biologically related types have been merged into broader categories.

15.2 Manual subclustering and curation

Heterogeneous clusters are subclustered interactively. Each subpopulation is inspected with marker gene feature plots and `FindAllMarkers`, and its subclusters are reassigned to the appropriate labels. This step requires biological knowledge of the system and is performed manually.

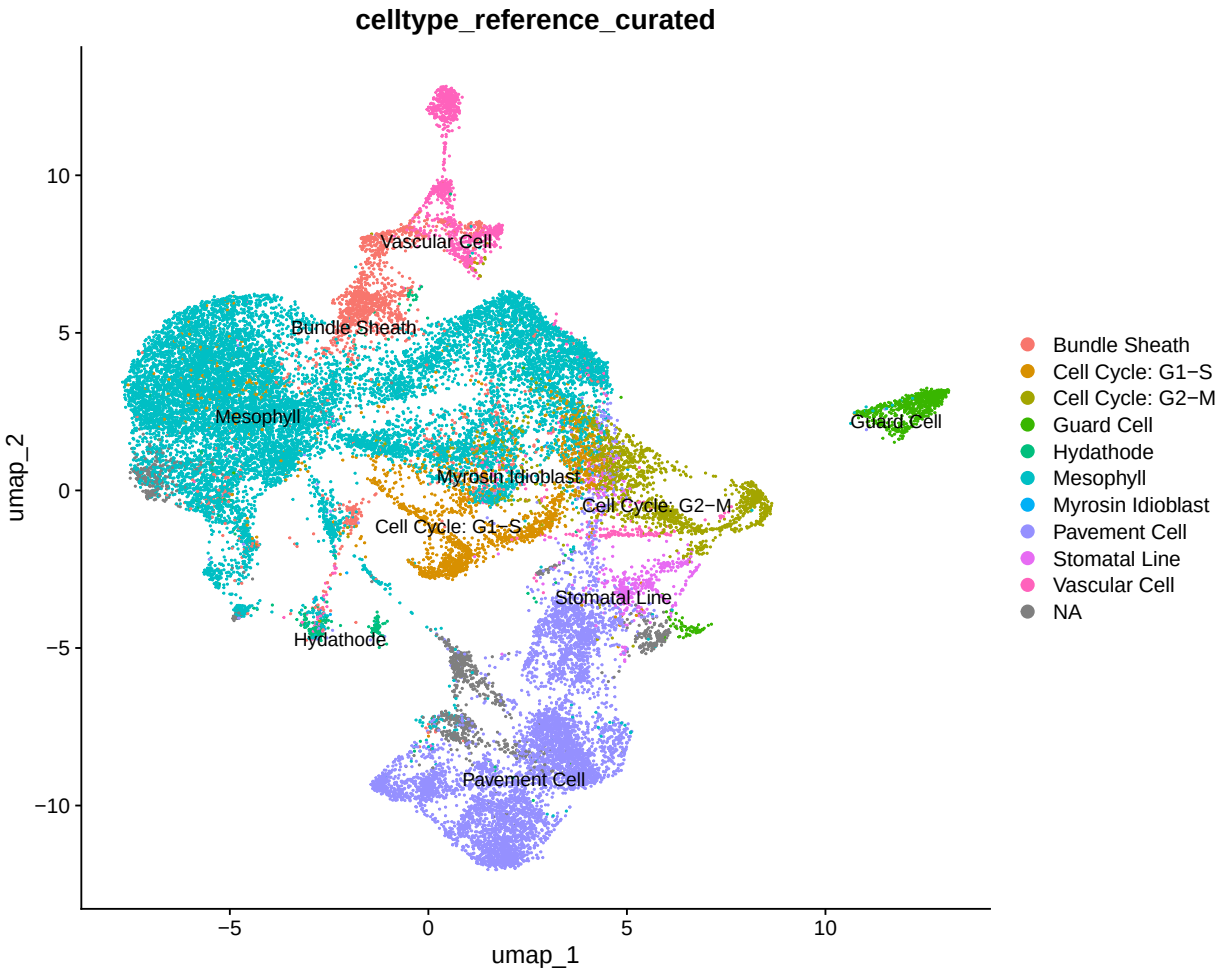

Figure 15: UMAP after manual curation. Each colour represents a curated cell-type label following subclustering and reassignment.

16 Step 14: Marker Gene Dot Plot

A dot plot summarises marker gene expression across all curated cell types simultaneously. Colour intensity encodes average normalised expression; dot size encodes the percentage of cells expressing the gene above zero. This visualisation facilitates rapid assessment of cell-type specificity for each marker.

```
## Error in `knitr::include_graphics()`:
## ! Cannot find the file(s): "../dotplot_marcadores.pdf"
```

---

17 Step 15: Export to Python (h5ad)

The curated Seurat object is exported to the AnnData format (`.h5ad`) for compatibility with the Python single-cell ecosystem (scanpy, scFates, palantir). All cell metadata, dimensionality reductions, and normalised counts are preserved.

```
# Export full object
exportar_para_scanpy(pbmc_harmony, "results/objs/pbmc_harmony_curated.h5ad")

# Export a single cell type
exportar_para_scanpy(
  subset(pbmc_harmony, subset = celltype_reference_curated == "Guard Cell"),
  "results/objs/GuardCell.h5ad"
)
```

---

18 Step 16: Pseudobulk per Cell Type and DESeq2 Differential Expression

18.1 Pseudobulk aggregation

Cells are split by curated cell type and pseudo-replicates are assigned within each type by randomly partitioning cells from the same condition. Counts are then summed across cells within each pseudo-replicate, producing a count matrix analogous to bulk RNA-seq. This aggregation is necessary to avoid the inflated statistical power that results from treating each cell as an independent replicate.

18.2 DESeq2 analysis

Pseudobulk count matrices are analysed with DESeq2, which fits a negative binomial model and applies empirical Bayes shrinkage of fold change estimates. All pairwise comparisons between conditions are run automatically and results are saved as CSV files (one per cell type per comparison), filtered at FDR $< 5\%$.

18.3 Volcano plots

Volcano plots visualise the DESeq2 output for each comparison. Each point represents one gene; the x-axis shows log2 fold change and the y-axis shows $-\log_{10}$(adjusted p-value). Significant and strongly changed genes (FDR < 0.05, $|\text{log2FC}| > 1$) are highlighted and labelled. Two cell types are displayed per page.

18.4 Differential expression heatmap

A log2 fold change matrix (genes $\times$ cell types) is constructed by aggregating results across all cell types for a given comparison and visualised as a hierarchical clustering heatmap. This reveals co-regulated gene sets across multiple cell types and highlights cell-type-specific responses.

---

19 Step 17: GO Enrichment Analysis

Gene Ontology (GO) enrichment analysis is performed on the differentially expressed genes from each comparison using `clusterProfiler`. Both the full and redundancy-reduced (via `simplify`) GO term sets are tested. Results are filtered to terms with adjusted p-value < 0.05 and visualised as dot plots showing the top enriched terms per cell type.

A pruning step further reduces GO term redundancy by retaining only terms above a minimum depth level in the GO hierarchy (default: level 6), ensuring that results represent specific biological processes rather than overly broad parent terms.

19.1 Organism configuration

The GO analysis is organism-specific. Change the following two parameters in `paperrr.R` to adapt to a different species:

Organism	`orgdb`	`keytype`
Arabidopsis thaliana	`org.At.tair.db`	`"TAIR"`
Human	`org.Hs.eg.db`	`"ENSEMBL"`
Mouse	`org.Mm.eg.db`	`"ENSEMBL"`

The background gene universe is derived automatically from the selected database using `keys(orgdb, keytype)`.

20 Adapting the Pipeline to Other Organisms

Three parameters in `paperrr.R` control the organism-specific parts of the analysis.

Mitochondrial / chloroplast gene patterns (QC section):

Organism	`mt_pattern`	`cp_pattern`
Arabidopsis thaliana	`"^ATMG"`	`"^ATCG"`
Human	`"^MT-"`	`NULL`
Mouse	`"^mt-"`	`NULL`

GO enrichment database (GO ENRICHMENT section):

```
# Arabidopsis
orgdb <- org.At.tair.db;  keytype <- "TAIR"

# Human
orgdb <- org.Hs.eg.db;    keytype <- "ENSEMBL"
```

DESeq2 comparisons — edit the `comparaciones` list to match your condition names:

```
comparaciones <- list(
  list(conds = c("Control", "Treatment"), tag = "ctrl_vs_treat"),
  list(conds = c("Control", "HighDose"),  tag = "ctrl_vs_high")
)
```

21 Output File Summary

File	Description
qc_prefilter.pdf	QC violin plots before filtering
qc_postfilter.pdf	QC violin plots after filtering
umap_preharmony.pdf	UMAP before batch correction
elbow_plot.pdf	K-means WSS elbow plot
clustree.pdf	Resolution sweep visualisation
umap_postharmony.pdf	UMAP after Harmony, by sample
umap_seuratclusters.pdf	UMAP coloured by cluster
conteocelulas.pdf	Cell count by sample per cluster
conteocelulas_seurat.pdf	Cell count by cluster
pseudobulk_correlation.pdf	Replicate Pearson correlation heatmap
FindAllMarkers.tsv	Marker gene table
clustree_annotated.pdf	Resolution sweep with cell-type labels
umap_annotated.pdf	UMAP with grouped cell-type labels
umap_curada.pdf	UMAP after manual curation
dotplot_marcadores.pdf	Marker gene dot plot
results/pseudobulk_replicas/*.csv	Pseudobulk count matrices per cell type
results/deseq2/<tag>/*.csv	DESeq2 results per cell type and comparison
results/volcano_plots/*.pdf	Volcano plots per comparison
results/diff/<tag>/heatmap_*.pdf	Log2FC heatmaps
results/Enrichment/<tag>/*.pdf	GO enrichment dot plots
results/objs/*.h5ad	Exported AnnData objects